

## Part 1: Registration Test Cases

### Test Case ID: TC-REG-001

- **Test Objective:** Verify user can register with valid information.
- **Preconditions:** The PostgreSQL database is running, and the email `newuser@example.com` does not exist in the `users` table.
- **Test Procedure:**
  1. Navigate to the Registration page.
  2. Enter a valid Full Name, `newuser@example.com`, and a valid password.
  3. Click "Register".
- **Expected Result:** The server returns a 201 Created status. A new user record is inserted into the PostgreSQL database. The React frontend redirects the user to the Login page (or logs them in automatically).
- **Testing Type:** Functional

### Test Case ID: TC-REG-002

- **Test Objective:** Verify registration fails when using an email that is already registered.
- **Preconditions:** A user with the email `existing@example.com` already exists in the `users` database table.
- **Test Procedure:**
  1. Navigate to the Registration page.
  2. Enter a valid Full Name, `existing@example.com`, and a password.
  3. Click "Register".
- **Expected Result:** The Express server catches the duplicate email and returns a 400 or 409 HTTP status. The React frontend displays a user-friendly error message (e.g., "Email already in use"). The app does not crash.
- **Testing Type:** Functional

### Test Case ID: TC-REG-003

- **Test Objective:** Verify SQL injection prevention on the registration form.
- **Preconditions:** The backend is running.
- **Test Procedure:**
  1. Navigate to the Registration page.
  2. In the "Full Name" field, enter: `Robert'); DROP TABLE users;--`
  3. Enter a valid email and password.
  4. Click "Register".
- **Expected Result:** The raw parameterized SQL queries correctly escape the input. The user is made with the literal name `Robert'); DROP TABLE users;--` and the database structure remains perfectly intact.
- **Testing Type:** Security

---

## Part 2: Login Test Cases

### Test Case ID: TC-LOG-001

- **Test Objective:** Verify successful login issues a valid JWT and handles frontend state.
- **Preconditions:** A user with email `test@example.com` and password `password123` exists.
- **Test Procedure:**
  1. Navigate to the Login page.
  2. Enter `test@example.com` and `password123`.
  3. Click "Login".
- **Expected Result:** The server returns a 200 OK with a JWT payload. The React frontend decodes the token using `jwt-decode`, saves the raw token to `localStorage("token")`, updates the authentication state, and redirects to `/my-listings`.
- **Testing Type:** Functional

### Test Case ID: TC-LOG-002

- **Test Objective:** Verify login gracefully handles incorrect passwords.
- **Preconditions:** A user with email `test@example.com` exists.
- **Test Procedure:**
  1. Navigate to the Login page.
  2. Enter `test@example.com` and `WrongPassword!`.
  3. Click "Login".
- **Expected Result:** The server returns a 401 Unauthorized status. No token is provided. The frontend displays an error message like "Invalid credentials" without exposing backend database information.
- **Testing Type:** Functional

**Test Case ID:** TC-LOG-003

- **Test Objective:** Verify route protection triggers when a user tries to access a protected route without logging in.
- **Preconditions:** The user is logged out (no token in `localStorage`).
- **Test Procedure:**
  1. Open the browser.
  2. Manually type the URL for the protected route:  
`http://localhost:3000/my-listings`.
  3. Press Enter.
- **Expected Result:** The React Router detects the missing authentication state and immediately redirects the user back to the Login page.
- **Testing Type:** Functional / Security

**Test Case ID:** TC-LOG-004

- **Test Objective:** Verify backend authorization middleware rejects invalid or tampered JWTs.
- **Preconditions:** The user has an API testing tool (like Postman) open.
- **Test Procedure:**
  1. Open Postman.
  2. Make a `GET` request to `/api/property/my-listings`.
  3. Set the Header to `Authorization: Bearer fake.tampered.token`.
  4. Send the request.
- **Expected Result:** The custom backend `authorize` middleware fails to verify the signature and returns a 401 Unauthorized or 403 Forbidden status. No private data is returned.
- **Testing Type:** Security